Introduction

Everyone is present
Every now puts their devices away for clear communication\

Team roles:

- Griffin, (Notetaker): Frontend and linking between front and backend.
- Amanda (Moderator): Team lead, worked on frontend, frontend to backend connections, and debugging
- Anitta: Worked on the backend, the database connections and frontend
- Nisa: worked on backend, frontend to backend connections
- Taqdeer: Testing, MCP, API endpoints

Now we are going to categorize each of the ADRs on the board

Criteria
We are assuming “AI Use” means how much AI was used during the implementation
We are assuming "appropriateness" means how well it worked out during implementation (lack of conflicts or regrets)

ADR TypeScript + React for Frontend
Many of us didn’t interact much with the Frontend, so they took an assumption

Universally, we all had the appropriateness as very appropriate and AI use as very high
We decided this because we never felt like using these had a tradeoff or held us back at any point
There wouldn’t have been much of a difference in choosing javaScript
Additionally, react made the front end really easy to duplicate for certain sections
Typescript was also very compatible with react
Since AI did most of the coding the language didn’t affect it very much
React gave us a good starting point
Typescript also made debugging easier
Backend was also in typescript so it was nice to have a compatibility

ADR PostgreSQL for DB

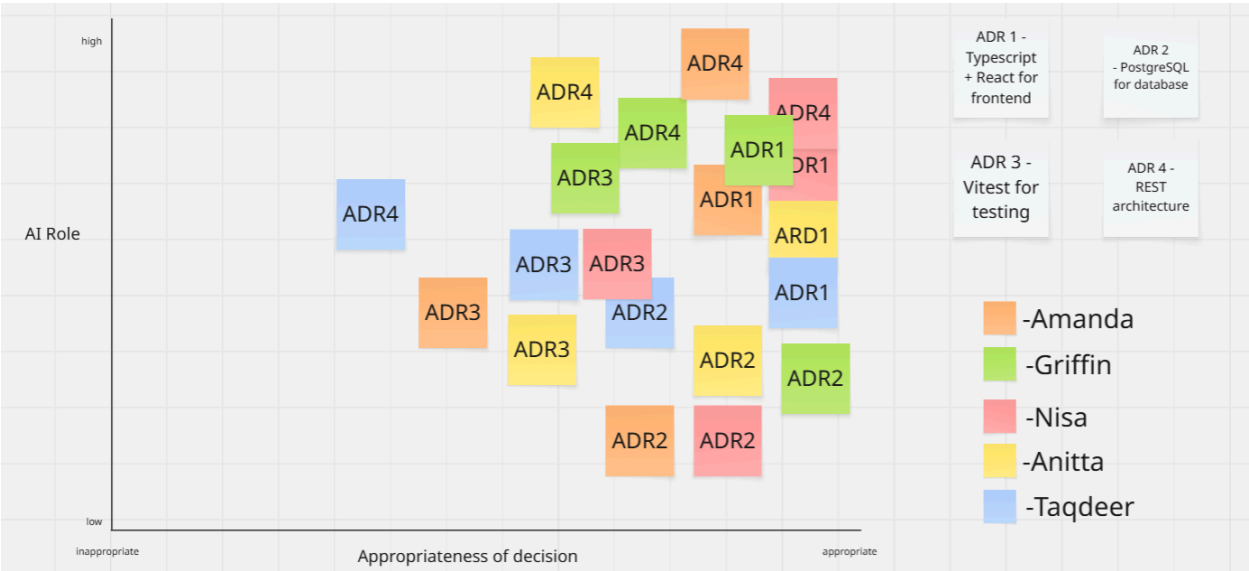
For this, most of us had Appropriate as high and AI as lower
We were able to create a schema for the DB before any actual implementation so a lot of the design was done by humans
PostgreSQL was familiar to a lot of us so it was easy for us to understand and implement without realying too much on AI
Most of the Frontend calls were easy to do without much AI use as well

ADR Vitest for Testing

Since Testing was done solely by one member, it was hard for a lot of us to say much regarding placement
The testing member, used AI when stuck but tried to learn as much as possible
This member was able to do a lot because she was familiar with the tool

<u>ADR REST Architecture</u>
This was the easiest one to do without much web development experience, so it was helpful to us
We used AI a lot initially to understand structure of calls, but moved away as we grew familiar
When looking at the board we noticed that all of us had put ADR4 as appropriate except one member
She found that the ADR made it difficult for the testing portion

<u>Board Discussion</u>



We noticed that there are not many sticky notes with an "inappropriate" scale
We noticed that the sticky notes tended to trend towards both high AI and high appropriateness. This was probably because many of us found that any issues that would have caused difficulties in ADR were handled completely by the AI.
We also did lots of research initially to find which decisions would benefit us the most and which we were most familiar with and would cause the least amount of potential conflicts.

<u>Follow-Ups</u>
If we had to design another software system, what would change in how we're using AI

-We would like to use it less, at least initially to build a proper foundation
-AI struggles with foundational code and because we were inexperienced it was hard for us to begin
-The AI struggled with debugging, so a lack of human understanding made debugging even harder

Do you think our team's usage of AI has helped our hindered our learning

-Griffin: Helped with some issues where I had little experience and getting an initial foothold. But, overall I do not think I could replicate any amount of my code now since AI had done most of it. Even if I wanted to avoid AI usage, the project was too big to try and teach myself concepts in time

-Amanda: I think that the usage of AI has mostly hindered our learning in this course. The main reason why is probably because we didn't have any base knowledge to go off of. It is very difficult to know what to do without having at least something to go off of, and in this case I think we ended up leaving a lot of the design decisions to the AI by accident because we didn't have a good grasp on what was going on. Also, I think if we were able to make a smaller scale project without the use of AI I would have learned a lot more. Generally I was not happy with how I used AI in this course.

-Anitta: AI initially helped with designing the structure of the files. I learned more about docker and how the API things work. AI definitely helped me to learn about the API keys and how it works, but when it comes to the codes, AI did most of it that I wasn't able to learn about the programming part of it.

-Nisa: Helped understand what files were necessary for the backend to work, and what overall project structure should look like. I learnt a little about typescript, but most of the backend code was done by AI. I would not have been able to complete the project without AI, but I did not learn much at the end of the project

-Taqdeer: It helped me clear specifications, gave me ideas and examples for code, which allowed me to personally connect with the work I wanted to get done.

Did we have any explicit or implicit rules on how to use AI for these projects? Since we did not, what rules would we add?

-Griffin: Don't use AI to create the backend schema. While this wasn't an issue for us, I would recommend it to any future groups as it allowed us to have a well thought-out and consistent database

-Amanda: Don't use AI to write any files from scratch, only to make files based off of previously handwritten files.

-Anitta: AI sometimes tries to change the current functions when we try to integrate new features. Be aware of it.

-Nisa: Have a foundational understanding of how to program the project without the use of AI, and use AI only when encountering especially tedious or difficult parts of the project.

-Taqdeer: Check that AI-generated code is correct and safe before using it.